campusEventHUB

External SOAP Partner Integration — Design-Time Contract

Team ID	TEAM-ID-HERE
Team Leader	NAME / ROLL NO. — REPLACE BEFORE SUBMISSION
Member 2	NAME / ROLL NO. — REPLACE BEFORE SUBMISSION
Member 3	NAME / ROLL NO. — REPLACE BEFORE SUBMISSION
Course / Assignment	Web Services — SOAP Partner Integration
Project	campusEventHUB — IIIT Vadodara

Before submission: replace Team ID and all member name/roll placeholders. This package is a design-time proposal based on the uploaded CampusHUB repository; it does not call a live SMS gateway. Partner URLs and credentials are illustrative.

1. Context and Partner Choice

Project context: The uploaded campusEventHUB SRS describes a campus event, club and activity platform. Students discover events and register using the current frontend's mock/local behavior; the backend README describes FastAPI as planned/future work. The current project documents do not identify an existing external SMS integration.

Proposed partner: a hypothetical external SMS gateway. **Operation:** `SendEventRegistrationSMS`, to send a registration confirmation after a student registers for an event such as HackIIITV 2026. This is a proposed partner edge, not a claim that SMS sending is already implemented.

CampusHUB can keep ordinary browser-to-application and planned internal APIs REST-based, while modeling this external notification-provider edge with SOAP because a formal WSDL gives both organizations a strict request/response contract. The partner boundary can use an agreed credential header and structured faults, making integration behavior easier to review and audit. The contract also gives the consumer a stable message reference and provider receipt ID for tracking submissions. The actual repository currently uses mock/local frontend data and does not yet implement this live gateway.

2. WSDL Contract

The attached partner.wsdl is a WSDL 1.1 contract and contains all required sections: **types** (XML Schema elements for request, response and credentials), **message** (input/output/header messages), **portType** (operation), **binding** (document/literal SOAP over HTTP and SOAPAction), and **service/port** (endpoint).

```
Operation: SendEventRegistrationSMS\nSOAPAction: https://contracts.campushub.example/sms/SendEventRegistrationSMS\nEndpoint: ht
```

3. HTTP Binding — Hand-Formed Request

```
POST /soap/SmsService HTTP/1.1\nHost: sandbox.sms-partner.example\nContent-Type: text/xml; charset=utf-8\nSOAPAction: "https://
```

Full endpoint URL: https://sandbox.sms-partner.example/soap/SmsService. The complete request places PartnerCredentials in the SOAP Header and SendEventRegistrationSMS in the Body.

4. SOAP Response and Fault

soap-response.xml represents an accepted submission and returns the message reference, provider message ID, ACCEPTED status and acceptance timestamp. ACCEPTED means the provider accepted the submission; it does not guarantee delivery to the handset. soap-fault.xml shows a realistic temporary provider outage. The SOAP Fault contains a partner-specific code PROVIDER_UNAVAILABLE and retryable=true. These are authored sample messages, not live gateway responses. The WSDL defines the successful input/output shapes and the XML shows the standard SOAP 1.1 fault envelope.

5. Discovery — Catalogue / Registry Entry

The integration owner would obtain the WSDL from the SMS provider's trusted developer portal or an approved internal service catalogue. A catalogue record points to the provider business, service, endpoint and WSDL contract (a tModel-style pointer); no UDDI server is run. Before using it, the owner would verify the provider and contract version, validate the endpoint and namespaces, and review any contract changes.

Field	Value
business	CampusHUB SMS Partner Sandbox (illustrative provider)
service	CampusHUBSmsPartnerService
endpoint	https://sandbox.sms-partner.example/soap/SmsService
wsdl / tModel-style pointer	https://contracts.campushub.example/contracts/partner.wsdl
binding / operation	SOAP 1.1 over HTTP; SendEventRegistrationSMS

The .example domains are reserved illustrative placeholders and will not resolve as live services.

6. Fault Mapping into CampusHUB Contract

The partner's fault vocabulary must not leak into student-facing responses. PROVIDER_UNAVAILABLE maps to a CampusHUB-owned application error such as `NOTIFICATION_TEMPORARILY_UNAVAILABLE`. Since the uploaded SRS and backend README do not define a public error schema for SMS notifications, this is a proposed internal contract mapping and should be aligned with the eventual API contract. The event registration itself should remain successful if registration was saved; SMS failure should be logged and retried asynchronously rather than falsely telling the student that event registration failed.

Partner SOAP fault	CampusHUB handling
PROVIDER_UNAVAILABLE; retryable=true	NOTIFICATION_TEMPORARILY_UNAVAILABLE (internal adapter code)
Provider message details	Do not expose raw SOAP fault to student
Notification submission failed	Queue a bounded retry; do not duplicate event registration
Student-facing behavior	Keep registration result independent; show registration confirmation in app

Submission checklist: integration.pdf (fill team details), partner.wsdl, soap-request.xml, soap-response.xml, soap-fault.xml. Commit all five required files into the repository required by the instructor and submit its link.